📄 Technical Specification & Environment Setup

Phase 1 | Days 3 & 4: Project Architecture & Dependency Isolation
Author: Jainish Mehta
Status: Executed & Verified

🎯 1. Executive Summary

The objective of Days 3 and 4 was to establish a scalable Machine Learning folder architecture and resolve the software engineering dilemma known as "Dependency Hell." In production environments, different ML models require strict, conflicting versions of libraries (e.g., Pandas, PyTorch). By implementing Python Virtual Environments (`venv`) and generating dependency manifests (`requirements.txt`), we ensured that the project remains completely isolated, deterministic, and highly portable across different operating systems.

🏗️ 2. Standardized Project Architecture

2.1 The "Cookiecutter" Directory Structure

- **Action:** Utilized the CLI to construct a modular directory tree (`data/` , `notebooks/` , `src/` , `models/`).
- **Engineering Rationale:** Beginners dump all scripts and datasets into a single root folder. Professional ML pipelines strictly separate concerns:
  - `data/` : Stores raw and processed datasets (never uploaded to remote VCS).
  - `src/` : Houses production-ready, object-oriented Python source code.
  - `notebooks/` : Reserved for experimental Jupyter notebooks and Exploratory Data Analysis (EDA).
  - `models/` : Stores serialized, trained ML models (e.g., `.pkl` or `.pt` files).

2.2 Forced Indexing via `.gitkeep`

- **Action:** Generated invisible `.gitkeep` files inside the empty architectural folders.
- **Engineering Rationale:** By default, the Git hashing algorithm completely ignores empty directories. Placing a `.gitkeep` file forces Git to track and push the structural blueprint to GitHub, ensuring that when another engineer clones the repository, the necessary folder hierarchy is already established.

🛡️ 3. Dependency Isolation & Virtualization

3.1 Python Virtual Environments (`venv`)

- **Action:** Executed `python -m venv venv` within the project root.
- **Engineering Rationale:**
  - **The Problem:** Installing Python packages globally (to the C: drive) creates version conflicts between concurrent projects.
  - **The Solution:** The `venv` module creates a localized, quarantined clone of the Python interpreter (`python.exe`) and the package manager (`pip`). Any library installed within this environment is sandboxed to this specific directory, leaving the global OS environment pristine.

3.2 Environment Activation & Path Routing

- **Action:** Executed `source venv/Scripts/activate`.
- **Engineering Rationale:** Running the `activate` script temporarily rewrites the system's `$PATH` environment variable. It explicitly instructs the terminal: *"Whenever the user types `python` or `pip`, intercept the command and route it to the local cloned directory instead of the global system directory."*

📦 4. Portability & Repository Integrity

4.1 Dependency Manifest Generation (`requirements.txt`)

- **Action:** Installed the `requests` library and executed `pip freeze > requirements.txt`.

- **Engineering Rationale:** A `venv` directory contains thousands of OS-specific binary files. A Windows `.exe` Python interpreter will fatally crash if downloaded onto a macOS or Linux machine.
  - Therefore, we *never* share the actual environment. Instead, `pip freeze` reads the environment's current state and generates a simple text-based manifest.
  - Another engineer can download this text file and run `pip install -r requirements.txt` to automatically fetch the correct OS-compatible binaries for their specific machine, ensuring cross-platform portability.

4.2 Version Control Exclusion (`.gitignore`)

- **Action:** Created a `.gitignore` file and appended `venv/` and `data/` to it.
- **Engineering Rationale:**
  1. **Bandwidth & Storage:** The `venv` directory is massive. Uploading it wastes bandwidth and clutters the commit history tree.
  2. **GitHub Limits:** GitHub imposes strict hard limits on repository sizes and blocks files over 100MB. ML datasets easily exceed gigabytes. The `.gitignore` file acts as a security firewall, instructing the Git tracking daemon to completely blind itself to datasets and environments, preventing accidental repository corruption.

📋 5. Command Glossary (For Quick Reference)

Command	Action Performed	System Impact
`python -m venv <name>`	Initialize Virtual Env	Clones the Python interpreter into a localized sub-directory.
`source <path>/activate`	Activate Environment	Rewrites the `$PATH` variable to prioritize the local Python binaries.
`pip freeze`	Snapshot Dependencies	Outputs an exact, deterministic list of installed packages and their version numbers.
`> requirements.txt`	Output Redirection	Catches the terminal output from `pip freeze` and writes it directly into a text file.